

Junior Rust Coder Manual

This document was extracted from project chat history. It captures real errors, fixes, and design decisions so junior Rust developers can avoid the same pitfalls and understand *why* things are done a certain way.

1. Project context

- **What this repo is:** A Tor v3 vanity `.onion` address generator. It uses **NVIDIA CUDA** and compiles a GPU kernel with Rust's **NVPTX** target (`nvptx64-nvidia-cuda`).
 - **Two crates:** A host binary (CLI, CUDA orchestration) and a `core` crate that is compiled to PTX and runs on the GPU. The host builds the kernel in `build.rs` by running a second `cargo` invocation for the kernel.
 - **Nightly required:** The NVPTX target and its features (`abi_ptx`, `stdarch_nvptx`) are unstable and only available on nightly. There is no stable path for this yet.
-

2. Edition and dependency hygiene

- Prefer **Rust 2021** (`edition = "2021"`). In 2021, `TryInto` / `TryFrom` are in the prelude, so you can remove `use core::convert::TryInto` in `no_std` crates to avoid "redundant import" warnings.
 - **Deprecated Cargo fields:** Avoid `authors` in `Cargo.toml`; it's deprecated. Keep `include` accurate (no placeholder paths like `"foo/bar"`).
 - **Error handling:** Use **anyhow** for application-level errors. The `failure` crate is unmaintained; migrate to `anyhow` and `anyhow::Result`.
 - **CLI:** Use **clap 4** with the **derive** API (`#[derive(Parser)]`). When you call `Args::parse()`, the **trait** that provides `parse` must be in scope: **use `clap::Parser`**; . Otherwise you get "no function or associated item named `parse` found for struct `Args`".
 - **cfg for NVPTX:** The only valid target arch for PTX is `nvptx64`. Use `target_arch = "nvptx64"` only (not `nvptx`); otherwise you get `unexpected_cfgs` warnings.
-

3. Build scripts (`build.rs`): avoiding deadlocks and wrong linkers

3.1 Cargo lock deadlock

If your `build.rs` runs **another** `cargo build` (e.g. to build a kernel), that inner `cargo` will try to lock the **same** `target/` directory. The **outer** `cargo build` already holds that lock until the build script exits. Result: **deadlock** (build appears stuck for hours).

Fix: Use a **separate target directory** for the inner build, e.g.

`CARGO_TARGET_DIR=target/nvptx-kernel` so the kernel build uses its own directory and its own lock.

3.2 PTX linker: don't use legacy `ptx-linker`

The old **`ptx-linker`** (and **`rustc-llvm-proxy`**) often fails on modern toolchains (e.g. "Unable to find lib target for 'llvm-sys' crate"). Prefer the **official** path:

- Use **`llvm-bitcode-linker`** (nightly component):

```
rustup component add llvm-bitcode-linker llvm-tools rust-src --  
toolchain nightly
```

- Build the kernel with **`cargo rustc`** and pass linker options directly:

```
-Z unstable-options -C link-self-contained=+linker -C linker-  
flavor=llbc
```

so `rustc` uses the bundled linker. The `nvptx64-nvidia-cuda` target defaults to a binary named `rust-ptx-linker`; passing `-C linker=/path` may **not** override that. Using `linker-flavor=llbc` avoids the PTX linker's different CLI (`--bitcode` vs `--target` / `--output`).

3.3 Large `Result` in build scripts

If your build script returns a large `Result` type, Clippy can warn about

`result_large_err`. Use a boxed error type, e.g. `Result<>`,

`Box<ptx_builder::error::Error>` (or whatever error type you use), so the return type is smaller.

3.4 Reproducible builds: one setup command

Document a **single one-liner** for toolchain setup so anyone can reproduce the build:

```
rustup install nightly && rustup target add nvptx64-nvidia-cuda --toolchain nightly && rustup c
```

If the kernel build fails (e.g. “Cargo.lock does not exist” / “unable to build with the standard library”), the compiler already suggests

`rustup component add rust-src`. Rely on that instead of reimplementing rustup’s checks in `build.rs` (output format and toolchain detection are fragile and caused false “missing rust-src” errors in the past).

4. `no_std` and panic handling

- **Unwinding:** Unwinding panics are **not** supported without `std`. For a `no_std` crate that is also built for the host, you must build with **`panic = "abort"`** when the standard library isn’t being rebuilt. Note: **`panic` cannot be set per-package** in `Cargo.toml`; it’s a **profile** setting. So you either set `panic = "abort"` for the whole project in `[profile.release]` and `[profile.dev]`, or you avoid defining a custom panic handler on the host.
 - **Duplicate lang item `panic_impl`:** If the same crate is built both as `no_std` (with its own `#[panic_handler]`) and linked into a `std` binary, you get “duplicate lang item: panic_impl” (one in `std`, one in your crate). **Fix:** Make `no_std` **conditional** on the target, e.g. `#![cfg_attr(target_arch = "nvptx64", no_std)]`, and only provide a panic handler for that target. On the host, the crate uses `std` and does not define a panic handler.
 - **Nightly feature names change:** The old `stdsimd` feature was removed. For PTX intrinsics (`core::arch::nvptx`, `_block_dim_x`, `_block_idx_x`, `_thread_idx_x`, `trap`), use `#![feature(stdarch_nvptx)]` (and keep `abi_ptx` as needed).
-

5. CUDA and cross-compilation

- **Why “CUDA not found” on the build machine:** (1) Only the **driver** is installed (runtime), not the **toolkit** (libs/headers for linking). (2) CUDA is in a non-default path (e.g. `/usr/lib/x86_64-linux-gnu` or `/opt`). (3) Building on a machine without CUDA (CI, laptop).
- **Make the lib path configurable:** Support **`CUDA_LIB_DIR`** (exact directory containing `libcuda.so`), and **`CUDA_HOME` / `CUDA_PATH`** (root; build uses `$VAR/lib64`). Add a fallback search path for common distro locations (e.g. `/usr/lib/x86_64-linux-gnu`, `/usr/lib/aarch64-linux-gnu`).

- **Cross-compilation:** For cross-compiled host targets (e.g. `x86_64` from an `aarch64` host or vice versa), set `CARGO_TARGET_<TRIPLE>_LINKER` to the appropriate cross-GCC and use a **separate** `CARGO_TARGET_DIR` (e.g. `target/``amd64` , `target/arm64` ) so host and kernel builds don't clash. For CUDA libs on the **target** side, use `X86_64_CUDA_LIB_DIR` / `AARCH64_CUDA_LIB_DIR` (or similar) so the build can point to target-arch CUDA libs.
-

6. Makefile and setup

- **Portable paths:** Prefer repo-relative defaults (e.g. `DEB_DIR ?= $(CURDIR)/../deb`) instead of hardcoded `$(HOME)/...` so the same Makefile works on any machine.
 - **Idempotent setup:** A `setup` target that runs `rustup install nightly` , `rustup target add ...` , and `rustup component add ...` is idempotent and safe to run every time. Making the main build depend on `setup` (e.g. `amd64: setup`) ensures deps are installed before build. Don't rely on fragile "verification" that parses rustup output (e.g. requiring the word "installed" in lines that don't contain it).
 - **CUDA check:** A `check-cuda` target that looks for `libcuda.so` in `CUDA_LIB_DIR` or a fallback path and exits with a clear message (and optional `make install-cuda` hint) helps users fix link errors quickly.
-

7. Troubleshooting quick reference

Symptom	Likely cause	What to do
<code>Cargo.lock</code> does not exist / "unable to build with the standard library"	Missing rust-src for nightly	Run the one-time setup (Installation in README); <code>rustup component add rust-src --toolchain nightly</code>
Build "stuck" at <code>tor-v3-vanity(build)</code>	Either slow first kernel build (many minutes) or deadlock (inner cargo	Use a separate <code>CARGO_TARGET_DIR</code> for the kernel build; run with <code>-vv</code> to see progress

Symptom	Likely cause	What to do
	locking same target)	
<code>linker rust-ptx-linker not found</code>	Target expects that binary; we use <code>llvm-bitcode-linker</code>	Don't install <code>ptx-linker</code> . Use <code>nightly</code> and <code>linker-flavor=llbc + link-self-contained=+linker</code> ; ensure components are installed
<code>unknown feature stdsimd</code>	Feature renamed/ removed in newer <code>nightly</code>	Use <code>stdarch_nvptx</code> for PTX intrinsics
<code>duplicate lang item panic_impl</code>	Crate defines panic handler and is also linked with <code>std</code>	Make <code>no_std</code> conditional (<code>cfg_attr(target_arch = "nvptx64", no_std)</code>) and remove host panic handler
<code>no function or associated item named parse found for struct Args</code>	<code>clap's Parser</code> trait not in scope	Add <code>use clap::Parser;</code>
<code>panic may not be specified in a package profile</code>	Cargo doesn't allow <code>panic</code> per-package	Set <code>panic = "abort"</code> in <code>[profile.release]</code> / <code>[profile.dev]</code> at the workspace/ root level if you need it
<code>unable to find library -lcuda / -lcudart</code>	CUDA libs not on default path	Set <code>CUDA_LIB_DIR</code> or <code>CUDA_HOME</code> ; or install toolkit (<code>make install-cuda</code> on Debian/Ubuntu)
Pre-flight "missing rust-src" even after <code>make setup</code>	Bug in <code>build.rs</code> (wrong toolchain from <code>CARGO</code> , or parsing "installed")	Prefer no custom pre-flight; let the kernel build fail with the compiler's own message. If you keep a check, use toolchain nightly explicitly and require <code>rustup</code> exit status success before parsing

8. Summary of lessons

1. **Traits in scope:** Derive macros don't auto-import traits; put `use clap::Parser` (and similar) where you call the methods.
2. **Build script nested cargo:** Use a **separate** `CARGO_TARGET_DIR` to avoid deadlock.
3. **PTX:** Use nightly, llvm-bitcode-linker, and `linker-flavor=llbc`; don't depend on the legacy ptx-linker.
4. **no_std + host:** Use **conditional** `no_std` and panic handler only for the GPU target to avoid duplicate lang items.
5. **Rustup checks in build.rs:** Prefer not reimplementing them; compiler/rustup errors are clear enough. If you do, use a fixed toolchain (`nightly`) and check command success before parsing.
6. **CUDA:** Make lib path configurable and document why "CUDA missing" can happen (driver vs toolkit, path, different machine).
7. **Reproducibility:** One setup command, idempotent Makefile setup, portable paths.

Generated from project agent transcripts. For build and usage details, see the main [README](#) and [CLAUDE.md](#).